The purpose of this notebook is to illustate an assignment that was part of the "Computational Programming with Python - NUMA01" course and is the combined work of students Anna Bernardi and Liam Kniberg.

This notebook is also available at https://github.com/oxpotassio/uni_bern02

Requirements

The dependencies for this code are:

- python 3.12
- numpy 2.0
- matplotlib 3.10

Motivation for Version Pinning

Version pinning is important to ensure the reproducibility of this notebook's results.

Major version updates to the libraries introduce usually significant changes so running the code with newer major versions could lead to errors. To then minimize these risks it's recomended to use the versions included in the provide requirements.

LICENSE

Creative Commons Legal Code

CC0 1.0 Universal

```
CREATIVE COMMONS CORPORATION IS NOT A LAW FIRM AND DOES NOT PROVIDE
LEGAL SERVICES. DISTRIBUTION OF THIS DOCUMENT DOES NOT CREATE AN
ATTORNEY-CLIENT RELATIONSHIP. CREATIVE COMMONS PROVIDES THIS
INFORMATION ON AN "AS-IS" BASIS. CREATIVE COMMONS MAKES NO WARRANTIES
REGARDING THE USE OF THIS DOCUMENT OR THE INFORMATION OR WORKS
PROVIDED HEREUNDER, AND DISCLAIMS LIABILITY FOR DAMAGES RESULTING FROM
THE USE OF THIS DOCUMENT OR THE INFORMATION OR WORKS PROVIDED
HEREUNDER.
```

Statement of Purpose

The laws of most jurisdictions throughout the world automatically confer exclusive Copyright and Related Rights (defined below) upon the creator and subsequent owner(s) (each and all, an "owner") of an original work of authorship and/or a database (each, a "Work").

Certain owners wish to permanently relinquish those rights to a Work for the purpose of contributing to a commons of creative, cultural and scientific works ("Commons") that the public can reliably and without fear of later claims of infringement build upon, modify, incorporate in

other works, reuse and redistribute as freely as possible in any form whatsoever and for any purposes, including without limitation commercial purposes. These owners may contribute to the Commons to promote the ideal of a free culture and the further production of creative, cultural and scientific works, or to gain reputation or greater distribution for their Work in part through the use and efforts of others.

For these and/or other purposes and motivations, and without any expectation of additional consideration or compensation, the person associating CC0 with a Work (the "Affirmer"), to the extent that he or she is an owner of Copyright and Related Rights in the Work, voluntarily elects to apply CC0 to the Work and publicly distribute the Work under its terms, with knowledge of his or her Copyright and Related Rights in the Work and the meaning and intended legal effect of CC0 on those rights.

1. Copyright and Related Rights. A Work made available under CC0 may be protected by copyright and related or neighboring rights ("Copyright and Related Rights"). Copyright and Related Rights include, but are not limited to, the following:

i. the right to reproduce, adapt, distribute, perform, display, communicate, and translate a Work;

ii. moral rights retained by the original author(s) and/or performer(s);

iii. publicity and privacy rights pertaining to a person's image or likeness depicted in a Work;

iv. rights protecting against unfair competition in regards to a Work, subject to the limitations in paragraph 4(a), below;

v. rights protecting the extraction, dissemination, use and reuse of data in a Work;

vi. database rights (such as those arising under Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, and under any national implementation thereof, including any amended or successor version of such directive); and

vii. other similar, equivalent or corresponding rights throughout the world based on applicable law or treaty, and any national implementations thereof.

1. Waiver. To the greatest extent permitted by, but not in contravention of, applicable law, Affirmer hereby overtly, fully, permanently, irrevocably and unconditionally waives, abandons, and surrenders all of Affirmer's Copyright and Related Rights and associated claims and causes of action, whether now known or unknown (including existing as well as future claims and causes of action), in the Work (i) in all territories worldwide, (ii) for the maximum duration provided by applicable law or treaty (including future time extensions), (iii) in any current or future medium and for any number of copies, and (iv) for any purpose whatsoever, including without limitation commercial, advertising or promotional purposes (the "Waiver"). Affirmer makes the Waiver for the benefit of each member of the public at large and to the detriment of Affirmer's heirs and successors, fully intending that such Waiver shall not be subject to revocation, rescission, cancellation, termination, or any other legal or equitable action to disrupt the quiet enjoyment of the Work by the public as contemplated by Affirmer's express Statement of Purpose.

2. Public License Fallback. Should any part of the Waiver for any reason be judged legally invalid or ineffective under applicable law, then the Waiver shall be preserved to the maximum extent permitted taking into account Affirmer's express Statement of Purpose. In addition, to the extent the Waiver is so judged Affirmer hereby grants to each affected person a royalty-free, non transferable, non sublicensable, non exclusive, irrevocable and unconditional license to exercise Affirmer's Copyright and Related Rights in the Work (i) in all territories worldwide, (ii) for the maximum duration provided by applicable law or treaty (including future time extensions), (iii) in any current or future medium and for any number of copies, and (iv) for any purpose whatsoever, including without limitation commercial, advertising or promotional purposes (the "License"). The License shall be deemed effective as of the date CC0 was applied by Affirmer to the Work. Should any part of the License for any reason be judged legally invalid or ineffective under applicable law, such partial invalidity or ineffectiveness shall not invalidate the remainder of the License, and in such case Affirmer hereby affirms that he or she will not (i) exercise any of his or her remaining Copyright and Related Rights in the Work or (ii) assert any associated claims and causes of action with respect to the Work, in either case contrary to Affirmer's express Statement of Purpose.

3. Limitations and Disclaimers.

a. No trademark or patent rights held by Affirmer are waived, abandoned, surrendered, licensed or otherwise affected by this document.

b. Affirmer offers the Work as-is and makes no representations or warranties of any kind concerning the Work, express, implied, statutory or otherwise, including without limitation warranties of title, merchantability, fitness for a particular purpose, non infringement, or the absence of latent or other defects, accuracy, or the present or absence of errors, whether or not discoverable, all to the greatest extent permissible under applicable law.

c. Affirmer disclaims responsibility for clearing rights of other persons that may apply to the Work or any use thereof, including without limitation any person's Copyright and Related Rights in the Work. Further, Affirmer disclaims responsibility for obtaining any necessary consents, permissions or other rights required for any use of the Work.

d. Affirmer understands and acknowledges that Creative Commons is not a party to this document and has no duty or obligation with respect to this CC0 or use of the Work.

Logarithm approximation

The following code approximates the logarithm function using an itertative method as described in:

Carlson, B. C. (1972). An algorithm for computing logarithms and arctangents. Mathematics of Computation, 26(118), 543–549. https://doi.org/10.1090/s0025-5718-1972-0307438-2

The method is based on computing the arithmetic and geometric mean of two values a_i and g_i in the following way:

- With a given value $x>0$, we initialize $a_0=\frac{(1+x)}{2}$, $g_0=\sqrt{x}$;
- We iterate $a_{i+1}=\frac{a_i+g_i}{2}$ and $g_{i+1}=\sqrt{a_{i+1}\,g_i}$;
- $\frac{x-1}{a_i}$ can be then considered an approximation to $\ln(x)$

Task 1: The following function approximate the ln function by n steps using the iterative method cited above.

```
import numpy as np
import matplotlib.pyplot as plt

def approx_ln(x,n):
    '''
    The function takes in an x value and returns the approximated
ln(x) ≈ (x-1)/a_n
    after n iterations, as described in https://doi.org/10.1090/s0025-
5718-1972-0307438-2
    '''
    #initializing starting values
    a0 = (1 + x)/2
    g0 = np.sqrt(x)

    if x>0:
        for i in range(n+1):
            ai = (a0 + g0)/2
            gi= np.sqrt(ai * g0)

            a0 = ai
            g0 = gi
    else:
        raise Exception(f'{x} is not greater than x, the ln(x) is not
defined')

    return (x-1)/a0
```

Task 2: We then plotted the numpy.ln and our approx_ln for n iterations.

```
x_asse = np.linspace(0.1, 50, 100) #starting at 0.1 because ln(0) is
not defined
n = 7

#the approx_ln works with scalars so it needs to be vectorized to use
it with an array
vec_approx_ln = np.vectorize(approx_ln)


fig, ax = plt.subplots()
```

```
plt.title('ln(x) and approx_ln for n iterations')

for i in range(2,n+1):
    ax.plot(x_asse, vec_approx_ln(x_asse, i), label=f'n = {i}')



ax.plot(x_asse,np.log(x_asse), 'k--', label='ln(x)')



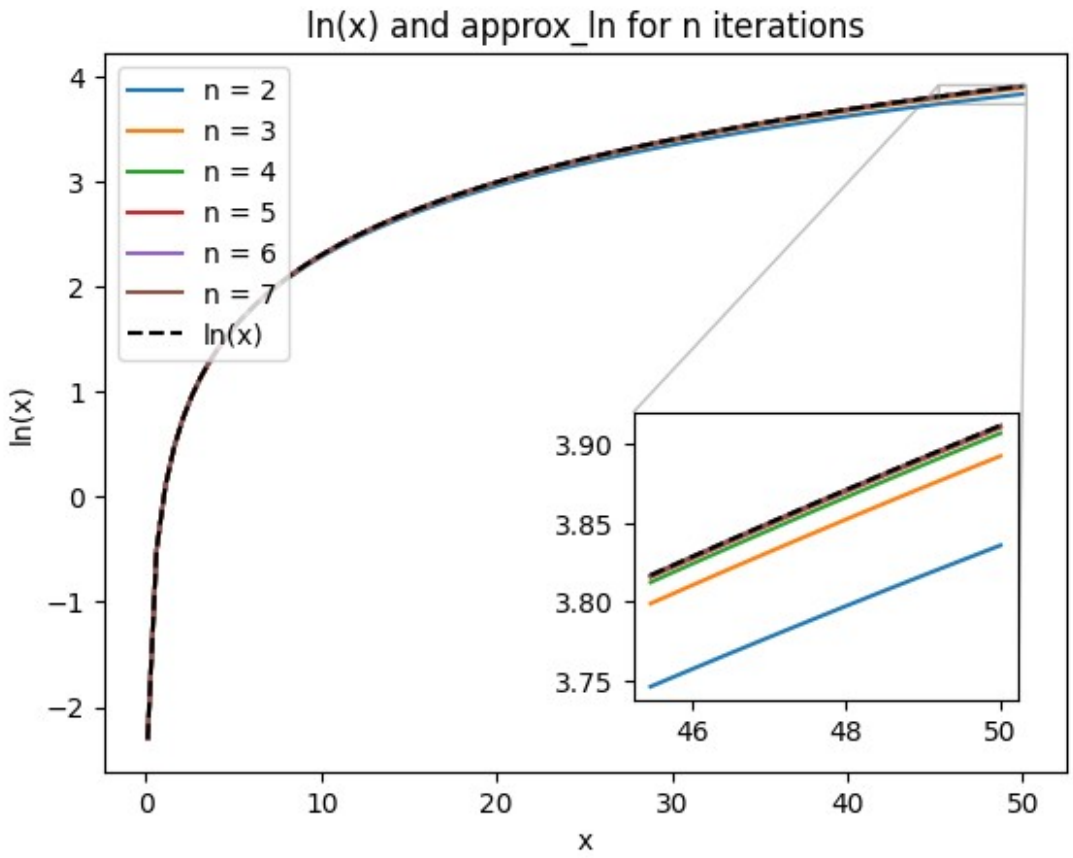
#zoomed in portion
x1 = x_asse[90:]

zoom = ax.inset_axes([0.55,0.1,0.4,0.4]) #loc and size

for i in range(2,n+1):
    zoom.plot(x1,vec_approx_ln(x_asse, i)[90:])

zoom.plot(x1, np.log(x_asse)[90:], 'k--')
ax.indicate_inset_zoom(zoom)

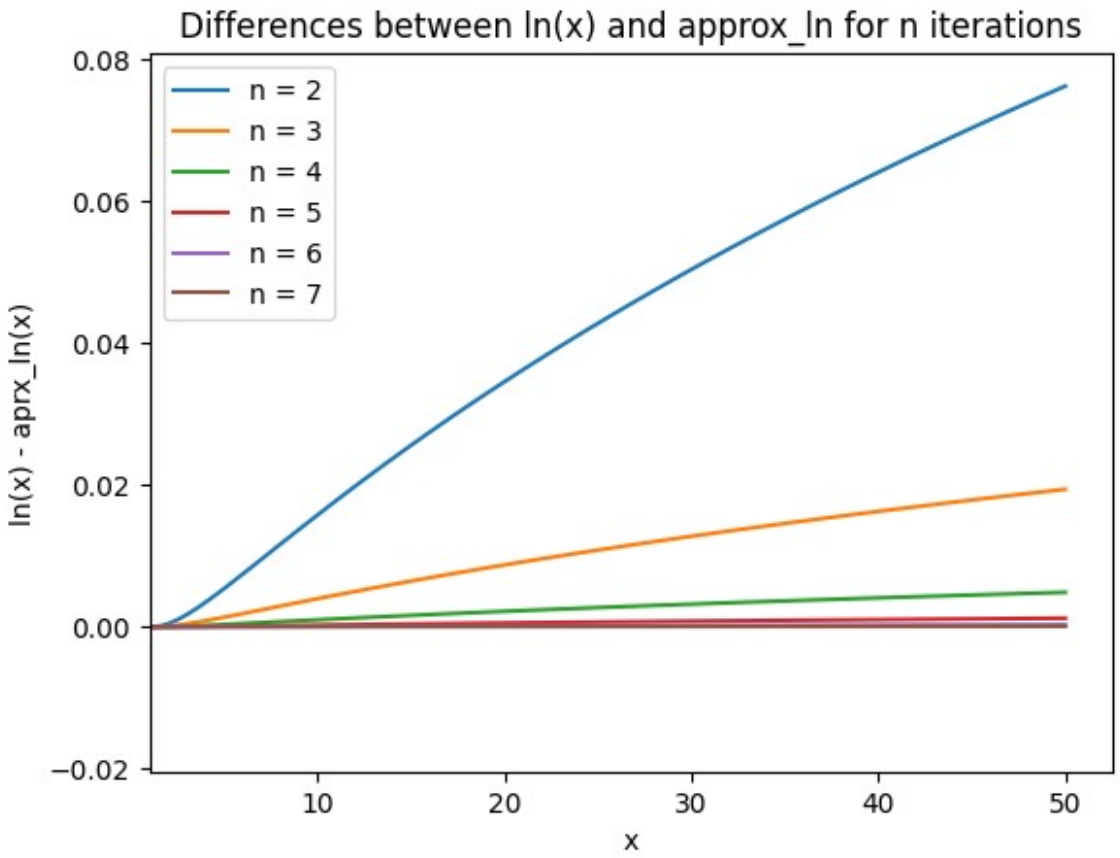
plt.xlabel('x')
plt.ylabel('ln(x)')
plt.legend(loc=2)
plt.show()
```



Task 3: We plotted the difference between the numpy.ln and the approximation

```
plt.figure()
for i in range(2,n+1):
    plt.plot(x_asse, np.log(x_asse)-vec_approx_ln(x_asse, i),
label=f'n = {i}')

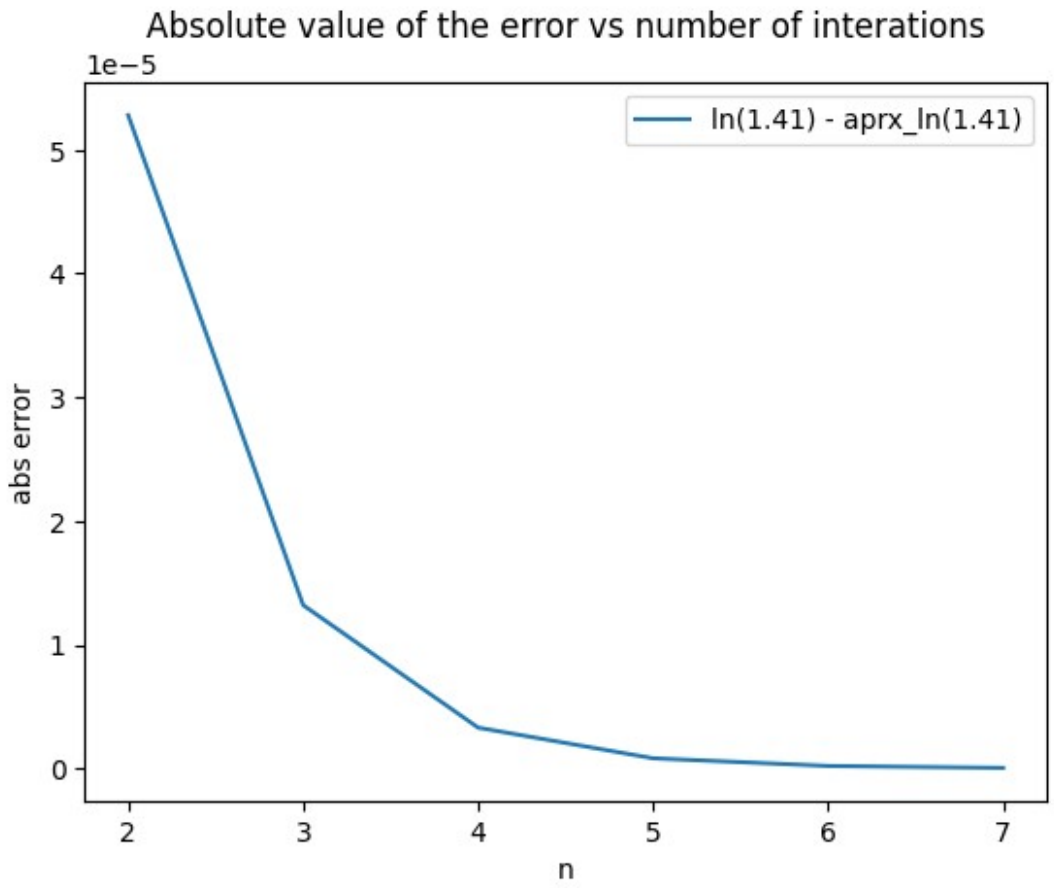

plt.title('Differences between ln(x) and approx_ln for n iterations')
plt.xlabel('x')
plt.xlim(1)
plt.ylabel('ln(x) - aprx_ln(x)')
plt.legend()
plt.show()
```



Task 4: Considering x = 1.41 we plotted the absolute value of the error versus the number of iterations n.

```
x = 1.41
error = []
for i in range(2,n+1):
    error.append(np.abs(np.log(x)-approx_ln(x, i)))


plt.plot(np.linspace(2,n,6), error, label='ln(1.41) - aprx_ln(1.41)')
plt.title('Absolute value of the error vs number of interations')
plt.xlabel('n')
plt.ylabel('abs error')
plt.legend()
plt.show()
```



Task 5: In the article cited above a method is suggested to accelerate the convergence and it involves for $i=0,\ldots,n$:

- $d_{0,i}=a_i$
- $d_{k,i}=\frac{d_{k-1,i}-4^{-k}d_{k-1,i-1}}{1-4^{-k}}$ with $k=1,\ldots,i$ for $i>0$

This new faster method then takes as an approximation for the logarithm the value $\frac{x-1}{d_{n,n}}$

The following function implements that approach

```
def fast_approx_ln(x,n):
    '''
    The function is the accelerated Carlsson method to approximate the
log.
    It takes a x value and returns the approximated ln(x) ≈
(x-1)/d_n_n
    after n iterations, as described in https://doi.org/10.1090/s0025-
5718-1972-0307438-2
    '''
    #initializing starting values
```

```
    a0 = (1 + x)/2
    g0 = np.sqrt(x)

    #and a list to store d[i][k]
    d = []
    #it's a list of lists where:
    #   d[i][0] is the apprx we'd get from approx_ln after i
iterations
    #   d[i][k] is the value at i-iteration and k-level

    for i in range(n+1):
        ai = (a0 + g0)/2
        gi= np.sqrt(ai * g0)

        a0 = ai
        g0 = gi

        #initial d[i][0] value
        d.append([ai])

        for k in range(1,i+1):
            dik = (d[i][k-1] - 4**(-k) * d[i-1][k-1]) / (1 - 4**(-k))

            d[i].append(dik)

    return (x-1)/d[n][n]
```

Task 6: the folllowing is an error plot for the faster aprx

```
x_asse = np.linspace(0.1, 20, 1000)

vec_fast_approx_ln = np.vectorize(fast_approx_ln)


plt.figure()


for i in range(2,n):
    error = np.abs(np.log(x_asse) - vec_fast_approx_ln(x_asse, i))

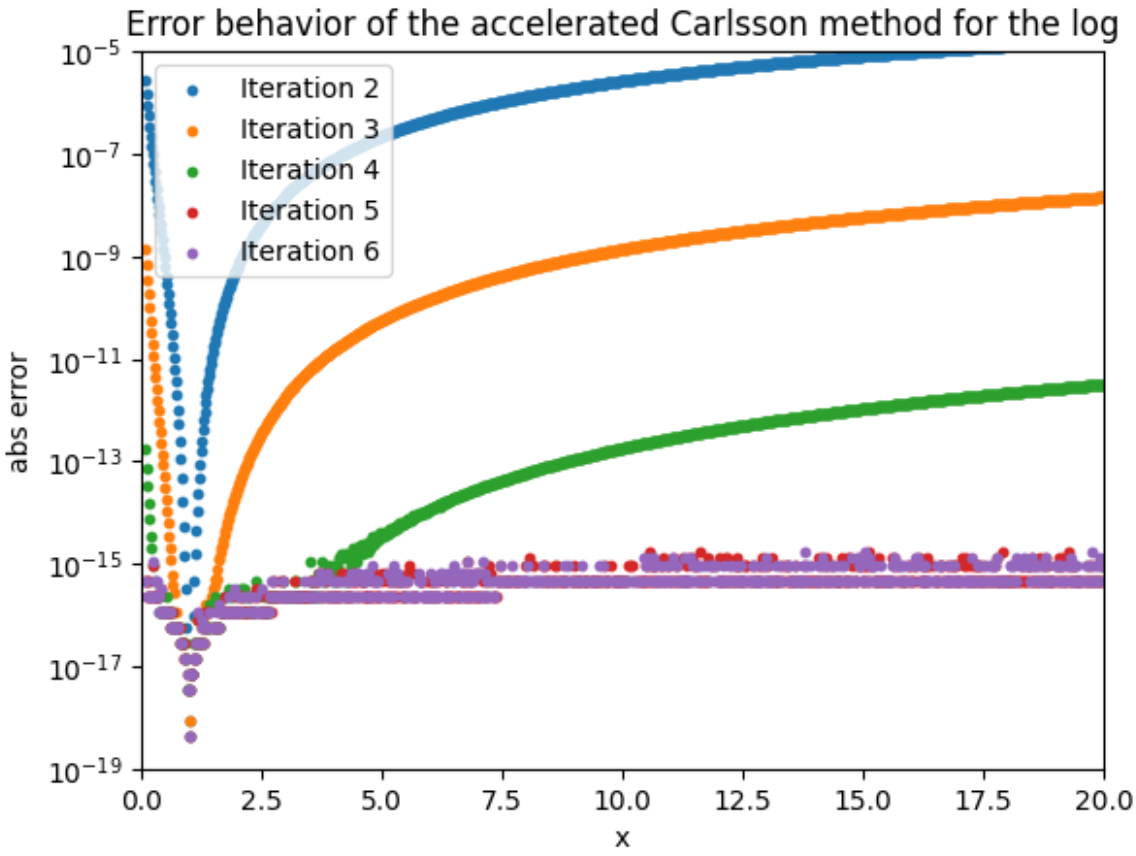
    plt.scatter(x_asse, error, label=f'Iteration {i}', s=10)



plt.title('Error behavior of the accelerated Carlsson method for the
log')
plt.xlabel('x')
plt.ylabel('abs error')
plt.yscale('log') #plotting the y axis in log scale to match the given
plot
```

```
plt.ylim(1e-19,1e-5)
plt.xlim(0.0,20.0)
plt.legend()


plt.show()
```



FAIR data principles

This notebook aims to set up a reproducible workflow following the FAIR data principles:

- FINDABLE: this notebook has been placed in a github repository which is indexed with relevant keywords. The underlying research that this notebook is based on is also findable.
- ACCESSIBLE: as a public repository, this notebook is freely available for anyone to access.
- INTEROPERABLE: the code is written in Python and it utilizes standard libraries like numpy and matplotlib, which are well-documented and interoperable with other tools and workflows. Also, the mathematical formulas are clearly presented.
- REUSABLE: a clear data usage license has been provided and so is the code, allowing others to replicate the results or adapt the methods.